Exercice 1 : ZoZoLivraison

On souhaite écrire un algorithme pour gérer les colis de la société de livraison **ZoZoLivraison**. Ce programme permet de suivre l'état des colis. Chaque colis est défini par :

- **id** : l'identifiant unique du colis.
- **poids** : le poids du colis en kilogrammes.
- **destination** : la destination du colis.
- **date** : la date d'envoi au format **DD/MM/YYYY**.
- **etat** : l'état du colis. Les états possibles sont : **"En préparation"**, **"En transit"** et **"Livré"**.

1. Donner la déclaration d'un colis

Définir un tableau **banqueColis** permettant de contenir **100** colis se trouvant à la gestion de **ZoZoLivraison**.

2. Écrire une procédure **saisirColis(banqueColis)** qui permet de saisir les informations des **100** colis.
3. Écrire une fonction **rechercherColis(id)** qui recherche un colis par son identifiant et retourne sa position dans la liste des colis. Si le colis n'est pas trouvé, la fonction retourne **-1**.
4. Écrire une procédure **afficherColisParEtat(banqueColis, etat)** qui affiche uniquement les colis ayant un état donné.
5. Écrire une procédure **afficherColisParDestination(dest)** qui affiche uniquement les colis ayant une destination donnée.
6. Écrire une procédure **afficherColisParDate(date)** qui affiche la liste des colis ayant une certaine date d'envoi.
7. Créer un menu pour votre application qui permet de sélectionner l'une des actions représentées par les fonctions et procédures écrites dans les questions précédentes.

Exercice 2 : Cotisations de parcelles

Soit un tableau **`tabCotisations`** comportant **`4`** lignes et **`10`** colonnes. Les **`4`** lignes correspondent aux quatre (**`4`**) trimestres de l'année et les **`10`** colonnes correspondent aux **`10`** parcelles d'une zone. Le tableau **`tabCotisations`** contient les cotisations des quatre (**`4`**) trimestres de chaque parcelle.

1. Écrire un algorithme permettant de saisir les données de cotisation de toutes les parcelles, de faire la somme de toutes les cotisations et d'afficher à la fin du traitement la somme totale calculée.
2. Écrire une procédure **`AffichageCotisation`** permettant d'afficher toutes les cotisations d'un trimestre (on demandera à l'utilisateur de saisir le trimestre).
3. Écrire une fonction nommée **`PlusGrandeCotisation`** qui retournera le montant le plus élevé des cotisations d'un trimestre (on demandera à l'utilisateur de saisir le trimestre).

4. Écrire une fonction nommée **PlusPetiteCotisation** qui retournera le montant le plus faible des cotisations enregistrées pour un trimestre (on demandera à l'utilisateur de saisir le trimestre).
5. Écrire une fonction nommée **MoyenneCotisations** qui retournera la moyenne des cotisations enregistrées pour un trimestre (on demandera à l'utilisateur de saisir le trimestre).
6. Écrire une procédure nommée **ListeDesCotisations** qui affiche la somme des cotisations obtenues par chaque parcelle pour les 4 trimestres.
7. Pouvez-vous calculer le chiffre d'affaires générer pour l'année à l'aide d'une action nommée ?

Exercice 3 : Employés et départements

Considérons les structures suivantes :
Date définie par les champs : **jour**, **mois** et **année**
Adresse définie par les champs : **Numéro**, **Rue**, **Commune**, **CodePostal**
Employe définie par les champs : **Nom**, **Prénom**, **Residence**, **DateDeNaissance**

1. Donnez la structure d'un enregistrement de type **Employe**.
2. Écrire une fonction nommée **AnneeNaissance(employe, date)** qui prend en paramètre un employé et une date, qui permet de vérifier si un employé est né avant une année donnée.
3. Écrire une fonction nommée **Commune** qui permet de vérifier si un employé est résidant dans une commune donnée. Vous déterminerez les paramètres de la fonction.
4. En utilisant les fonctions ci-dessus (appels), écrire un programme qui permet de
   a. Remplir un tableau appelé **departements** de **N** éléments de type **Employe**, avec $N \leq 100$.
   b. Afficher les noms et prénoms des employés résidents dans une commune qui sera donnée par l'utilisateur.
   c. Afficher les noms et prénoms des employés résidents dans une commune et nés avant une date donnée. Le nom de la commune et la date seront donnés par l'utilisateur.

Exercice 4 : Gestion des ventes d'un supermarché

Dans un supermarché un produit est caractérisé par :

- son code à barre (**50** chiffres) ;
- son nom (**20** caractères) ;
- et son prix.

Un caissier caractérisé par :

- son identifiant (**30** caractères) ;
- son nom (**20** caractères) ;
- l'ensemble des produits vendus pendant la journée (tableau de **100** produits). On pourra calculer le solde de sa caisse à l'aide de ce tableau de produit ;

Travail à faire :

1. Définir la structure de données « **Produit** ».
2. Définir la structure « **Caissier** ».

3. Créer un tableau nommé **ListeProduits** permettant de saisir **30** produits.
4. Créer un tableau nommé **ListeCaissier** permettant de saisir **10** caissiers.
5. Écrire une procédure nommée **AfficherProduit(produit)** permettant d'afficher un produit.
6. Écrire une fonction **VendreProduit(ListeProduit, caissier)** permettant de vendre un produit par un caissier. On retournera Vrai si la vente est possible et Faux dans le cas contraire.
7. Écrire une procédure nommée **AffichageListeProduits(ListeProduits)** qui permet d'afficher la liste des produits vendus par le caissier.
8. Écrire une fonction **SoldeCaisse(caissier)** qui retourne le solde final de la caisse à la fin d'une journée (la somme des prix des produits vendus par le caissier)
9. Écrire une fonction **ChangerPrix(ListeProduits, codebar, nouveauPrix)** permettant de changer le prix d'un Produit dans la liste des produits **ListeProduits**.
10. Écrire une fonction **PlusChere(ListeProduits, caissier)** qui retourne le produit le plus chère vendue par un caissier.
11. Écrire une fonction **ChercherProduit(codebar, caissier)** permettant de vérifier est-ce qu'un produit a été vendu ou non par un caissier. (La recherche s'effectuera suivant le code à barre du produit). On retournera Vrai si le produit existe et Faux dans le cas contraire.
12. Écrire une fonction nommée **MeilleurCaissier(ListeCaissier)** permettant de déterminer le meilleur caissier de tous les caissiers.
13. Dans votre programme principal :
    a. Écrire les instructions permettant de saisir les ventes des **30** caissiers ;
    b. Affichez la liste des produits vendus par les **30** caissiers ;
    c. Calculez et afficher le solde de tous les caissiers ainsi que le chiffre d'affaires global ;
    d. Afficher le produit le plus chère ;
    e. Demandez à l'utilisateur s'il veut changer le prix d'un produit. En cas de réponse positive, obtenir le code du produit en question et changez son prix ;
    f. Affichez le meilleur caissier.